

AtaxChile Backend

Sistema de Registro de Pacientes con Ataxia

REST API · NestJS · TypeScript · TypeORM · PostgreSQL

Objetivo del Sistema

Plataforma de gestión para la **Agrupación AtaxChile** que permite:

- Registro y seguimiento de **socios** (pacientes y representantes)
- Clasificación del **tipo de ataxia** por catálogo controlado
- Generación de **estadísticas agregadas** poblacionales en Chile
- Gestión de **usuarios administrativos** con control de acceso por roles
- Trazabilidad completa mediante **auditoría inmutable**

El sistema **no** contempla ficha médica, tratamientos ni prescripciones.

Stack Tecnológico

Capa	Tecnología
Framework	NestJS 11 + TypeScript
ORM	TypeORM 0.3
BD Producción	PostgreSQL (Supabase)
BD Desarrollo	PostgreSQL 16 vía Docker Compose (puerto 5434)
Infraestructura local	Docker Compose — PostgreSQL 16 (puerto 5434)
Autenticación	JWT (Passport) — access + refresh tokens

Capa	Tecnología
Seguridad HTTP	helmet (headers) + @nestjs/throttler (rate limiting)
Validación	class-validator + class-transformer
Email	Resend
Documentación	@nestjs/swagger + Swagger UI
Testing	Jest + Supertest

Arquitectura de Módulos

src/	
└─ auth/	Autenticación JWT, guards, decoradores
└─ users/	Usuarios administrativos (staff y directivos)
└─ geo/	Regiones y comunas de Chile (datos de referencia)
└─ miembros/	Socios de la agrupación
└─ ataxia-types/	Catálogo controlado de tipos de ataxia
└─ diagnostico-clinico/	Diagnóstico clínico por miembro (1:1, mutable)
└─ evaluacion-funcional/	Evaluaciones SARA y movilidad (append-only)
└─ stats/	Estadísticas agregadas (solo lectura)
└─ audit/	Registro de auditoría inmutable
└─ exports/	Exportación de datos (CSV/XLSX/PDF)
└─ mailer/	Envío de emails (Resend)
└─ middlewares/	Middlewares personalizados
└─ common/	TransformInterceptor, PaginatedResult, RutValidator

Separación clave: users (acceso al sistema) ≠ members (socios registrados)

Autenticación — Estrategia JWT dual

Token	Duración	Almacenamiento
access_token	15 minutos	Solo en cliente

Token	Duración	Almacenamiento
refresh_token	7 días	Hasheado en BD

El `refresh_token` se almacena **hasheado** en `users.refreshToken`. La app falla al iniciar si faltan `JWT_SECRET` o `JWT_REFRESH_SECRET`.

Control de Acceso — Roles (RBAC)

Rol	Permisos
SUPERADMIN	Bypass global — gestión completa del sistema y usuarios
ADMIN	Miembros, catálogos, reportes y lectura completa
SECRETARIO	Crear, actualizar miembros, diagnósticos y evaluaciones
TESORERO	Lectura de miembros, cuotas y estadísticas de resumen
USUARIO	Rol por defecto al registrarse — sin acceso administrativo

El primer usuario registrado recibe automáticamente el rol `SUPERADMIN`.
`SUPERADMIN` tiene `bypass global` en `RolesGuard` — acceso implícito a todos los endpoints.

Flujos de Autenticación

Login y sesión persistente

POST /auth/login	→ access_token (15 min) + refresh_token (7 días)
POST /auth/refresh	→ nuevo access_token sin volver a loguearse
POST /auth/logout	→ invalida el refresh_token en BD

Recuperación de contraseña

POST /auth/forgot-password → genera token, envía email (respuesta genérica)
POST /auth/reset-password → valida token, hashlea nueva contraseña con bcrypt
POST /auth/change-password → cambia contraseña conociendo la actual

Registro de Cuenta — Pasos 1 a 6

Ruta: POST /users/register (pública)

1. Cliente envía: nombre, email, password
↓
2. Backend verifica que el email no esté registrado
→ 409 si ya existe
↓
3. Hashea password con bcrypt (10 rounds)
↓
4. Asigna rol = 'usuario' automáticamente
(no se acepta rol en el body)
↓
5. Genera token de activación con randomUUID()
↓
6. Almacena token + expiración (24 h) en BD
(users.tokenActivacion)

Registro de Cuenta — Pasos 7 a 11

Ruta de activación: GET /users/activar/:token (pública)

7. Envía email con enlace: `/activar?token=<token>`
↓
8. Responde: "Hemos enviado un correo a {email}
para activar la cuenta"
↓
9. Usuario hace clic → GET `/users/activar/:token`
↓
10. Backend valida token + expiración
→ activa cuenta (`cuentaActivada = true`)
↓
11. Limpia `tokenActivacion` y `tokenExpiracion` de la BD

Hasta que la cuenta **no esté activada**, el login es rechazado.

Recuperación de Contraseña — Paso 1

Ruta: POST `/auth/forgot-password` (pública)

```
POST /auth/forgot-password { email }  
  → Busca usuario por email  
  → Genera token con crypto.randomBytes(32)  
  → Almacena HASH del token + expiración (1 hora) en BD  
  → Envía email con enlace al frontend:  
    /reset-password?token=<token-en-claro>  
  → Responde SIEMPRE con mensaje genérico  
    (no confirma si el email existe →  
    previene enumeración de usuarios)
```

Recuperación de Contraseña — Paso 2

Ruta: POST `/auth/reset-password` (pública)

```
POST /auth/reset-password { token, nuevaPassword }
  → Busca usuario por HASH del token recibido
  → Verifica que el token no haya expirado
    (400 si expiró)
  → Hashea la nueva contraseña con bcrypt (10 rounds)
  → Guarda nueva contraseña en BD
  → Limpia resetPasswordToken y resetPasswordExpires
  → Registra evento en auditoría
```

Módulo Geo

Regiones y Comunas de Chile

- Datos de referencia **estáticos** — 16 regiones + 346 comunas
- Poblados automáticamente con un **seeder** al iniciar la app (solo si las tablas están vacías)
- Al crear un miembro se envía `comunaId` ; la **región se obtiene** vía `comuna` → `region`

No se implementa `DELETE` — la eliminación rompería FKs de miembros existentes.

Módulo Geo — Endpoints

Método	Ruta	Acceso
GET	/geo/regiones	Público
GET	/geo/regiones/:id/comunas	Público
POST	/geo/regiones	superadmin, admin
PATCH	/geo/regiones/:id	superadmin, admin
POST / PATCH	/geo/comunas	superadmin, admin

Módulo Tipos de Ataxia

Catálogo Controlado

- **59 tipos** clasificados en 4 grupos
- Migrado desde el sistema legado SQL (`tipo_ataxia`) y enriquecido con terminología clínica actual

Grupo	Descripción	Ejemplos
hereditaria	Origen genético	Friedreich, SCA1–SCA30, DRPLA, FXTAS
adquirida	Causas externas	MSA-C, alcohólica, paraneoplásica, CANVAS
idiopatica	Sin causa identificada	SAOA, esporádica no clasificada
otra	En investigación	Origen combinado, sin diagnóstico

La confirmación diagnóstica (`genetico` / `clinico` / `probable`) pertenece a `DiagnosticoClinico` , no al catálogo.

Seeder de Tipos de Ataxia

Migración desde tabla SQL legacy (`tipo_ataxia`) al nuevo seeder TypeScript:

32 registros previos → **59 registros** en el seeder actualizado

Origen	Destino	Acción
SCA1 – SCA30 (con gaps)	HEREDITARIA	Incorporados con nombre clínico
EA1 – EA7	HEREDITARIA	Completados (SQL solo tenía 7)
DRPLA , FXTAS , MIRAS , MSS	HEREDITARIA	Incorporados con descripción
ATAXIA CEREBELOSA ADQUIRIDA	ADQUIRIDA	Incorporado
DESCONOCIDA	OTRA	Renombrado con descripción
MSA-C, CANVAS, AVED, etc.	Nuevo sistema	Agregados (no estaban en SQL)

Módulo Miembros

Perfiles diferenciados

Campo	Paciente	Representante
tipoAtaxiaId	Referencia al catálogo	null
diagnosticoClinico	Relación 1:1	null
tipoRepresentacion	null	Obligatorio
representadoId / representadoNombre	null	Al menos uno

El diagnóstico completo vive en DiagnosticoClinico , separado del miembro.

Módulo Miembros — Campos Clave (1/2)

Campo	Tipo	Descripción
rut	string único	RUT chileno validado
nombre	string	Nombre completo
celular	string nullable	Teléfono móvil
profesion	string nullable	Profesión u oficio
estadoCivil	enum	soltero / casado / viudo / divorciado

Módulo Miembros — Campos Clave (2/2)

Campo	Tipo	Descripción
estado	enum	activo / renunciado / suspendido / fallecido
fechaCambioEstado	Date nullable	Se actualiza automáticamente
representadoNombre	string nullable	Nombre del representado (si no está registrado)
representadoRut	string nullable	RUT del representado (si no está registrado)

`anio_inscripcion` y `anio_cambio_estado` no se almacenan — se derivan con `.getFullYear()` .

Módulo Diagnóstico Clínico

Entidad `DiagnosticoClinico` — relación 1:1 con `Miembro`

Campo	Tipo	Descripción
tipoAtaxiaId	FK → <code>AtaxiaType</code>	Tipo de ataxia del catálogo
subtipo	string nullable	Texto libre (ej: SCA2, Friedreich)
confirmacion	<code>ConfirmacionDiagnostico</code>	genetico / clinico / probable
fechaDiagnostico	date nullable	Fecha del diagnóstico
institucion	string nullable	Centro donde se diagnosticó
medico	string nullable	Médico tratante
observaciones	text nullable	Notas adicionales

Solo los pacientes (`esRepresentante: false`) tienen diagnóstico clínico asociado.

Módulo Evaluación Funcional

Entidad EvaluacionFuncional — append-only

Campo	Tipo	Descripción
fecha	date	Fecha de la evaluación
nivelMovilidad	NivelMovilidad	ambulatorio / silla parcial / silla total / postrado
puntuacionSara	int nullable	Escala SARA 0–40 (estándar internacional)
disartria	boolean	Dificultad para hablar
disfagia	boolean	Dificultad para tragar
nistagmo	boolean	Movimiento involuntario de ojos
tieneCuidador	boolean	Tiene cuidador asignado
registradoPorId	FK → User	Usuario que registró la evaluación

Append-only: sin UPDATE ni DELETE. Cada evaluación es un hecho histórico inmutable.

Representación de Socios

Un **representante** es un familiar, tutor o cuidador de una persona con ataxia.

```
export enum TipoRepresentacion {
  PADRE_MADRE = 'padre_madre',
  CONYUGE = 'conyuge',
  HIJO_HIJA = 'hijo_hija',
  TUTOR_LEGAL = 'tutor_legal',
  CUIDADOR = 'cuidador',
  OTRO = 'otro',
}
```

- Si la persona representada **está registrada** → representadoId (UUID, relación auto-referencial)

- Si **no está registrada** → representadoNombre + representadoRut (campos de texto)
- Los representantes se **excluyen** de las estadísticas de diagnóstico de ataxia

API REST — Endpoints por Módulo (1/2)

Módulo	Base	Operaciones
Auth	/auth	login, refresh, logout, forgot/reset/change-password
Usuarios	/users	CRUD + activar cuenta + cambio de rol/status
Geo	/geo	Regiones y comunas (lectura pública, escritura admin)
Tipos de Ataxia	/ataxia-types	Catálogo con filtro ?grupo= + soft delete
Miembros	/miembros	POST , GET , GET /:id , PATCH /:id , PATCH /:id/estado , PATCH /:id/vincular-usuario
Diagnóstico Clínico	/miembros/:id/diagnostico	POST , GET , PATCH
Eval. Funcional	/miembros/:id/evaluaciones	POST (append-only), GET , GET /ultima

Base URL: /api/v1

API REST — Endpoints por Módulo (2/2)

Módulo	Base	Operaciones
Estadísticas	/stats	resumen · miembros · diagnósticos · funcional · geográfico · cuotas*

Módulo	Base	Operaciones
Auditoría	/audit-logs	Solo lectura, acceso SUPERADMIN
Exportaciones	/exports	CSV / XLSX — solo roles autorizados

* *pendiente módulo cuotas*

Documentación Swagger

Configuración aplicada

```
// main.ts
const config = new DocumentBuilder()
  .setTitle('AtaxChile API')
  .setDescription('Sistema de registro de miembros de la Asociación AtaxChile')
  .setVersion('1.0')
  .addBearerAuth()
  .build();
SwaggerModule.setup('api/docs', app, document);
```

UI disponible en: <http://localhost:5000/api/docs>

Cobertura

- ✓ `@ApiProperty` en todos los DTOs con ejemplos reales
- ✓ `@ApiTags` agrupa endpoints por módulo
- ✓ `@ApiBearerAuth` en rutas protegidas
- ✓ `@ApiOperation` + `@ApiResponse` en cada endpoint
- ✓ `PartialType` migrado de `@nestjs/mapped-types` → `@nestjs/swagger`

Módulo Stats

Estadísticas agregadas — solo lectura

Endpoint	Roles	Descripción
GET /stats/resumen	ADMIN, SEC, TES	Totales y variaciones recientes
GET /stats/miembros	ADMIN, SECRETARIO	Distribución por estado
GET /stats/diagnosticos	ADMIN, SECRETARIO	Por tipo de ataxia y confirmación
GET /stats/funcional	ADMIN, SECRETARIO	Última evaluación por miembro
GET /stats/geografico	ADMIN, SECRETARIO	Distribución por región

- Sin entidad propia — agrega datos con `QueryBuilder + GROUP BY`
- Representantes excluidos de diagnósticos, funcional y geográfico

Módulo Audit

Registro inmutable de eventos del sistema

Campo	Tipo	Descripción
accion	enum	LOGIN · CREAR_MIEMBRO · MODIFICAR_MIEMBRO · ...
entidad	varchar	Tabla afectada (miembros , users , etc.)
entidadId	varchar null	ID del registro afectado
detalle	json null	Campos cambiados u otros datos del evento
usuarioId	FK null	Quién ejecutó la acción
ip	varchar null	IP del cliente

- **Append-only** — sin UPDATE ni DELETE

- `AuditService` es interno — los servicios lo inyectan; no hay endpoint de creación
- Fallo en auditoría no interrumpe la operación principal (fire-and-forget)
- Lectura restringida exclusivamente a SUPERADMIN

Módulo Audit — Endpoints

Método	Ruta	Acceso	Descripción
GET	<code>/audit-logs</code>	SUPERADMIN	Lista paginada con filtros
GET	<code>/audit-logs/:id</code>	SUPERADMIN	Detalle de un evento específico

Filtros disponibles en `GET /audit-logs` :

- `?page=1&limit=20` — paginación
- `?accion=LOGIN` — por tipo de acción
- `?entidad=miembro` — por entidad afectada
- `?usuarioId=5` — por usuario
- `?desde=2024-01-01&hasta=2024-12-31` — rango de fechas

Módulo Exports

Exportación de datos — CSV / XLSX / PDF

Campo	Tipo	Descripción
<code>formato</code>	enum	<code>csv</code> · <code>xlsx</code> · <code>pdf</code>
<code>estado</code>	string	Filtrar por estado del socio
<code>regionId</code>	number	Filtrar por región
<code>tipoAtaxiaId</code>	number	Filtrar por tipo de ataxia
<code>fechaDesde</code>	ISO date	Filtrar por fecha de ingreso

Campo	Tipo	Descripción
fechaHasta	ISO date	Filtrar por fecha de ingreso

Módulo Exports — Endpoints

Método	Ruta	Acceso	Descripción
GET	/exports/miembros	ADMIN, SECRETARIO, TESORERO	Exportar lista de miembros
GET	/exports/miembros/:id/evaluaciones	ADMIN, SECRETARIO	Ficha individual con historial (PDF)

Formatos disponibles: [?formato=csv](#) · [?formato=xlsx](#) · [?formato=pdf](#)

Columnas por rol:

- ADMIN / SECRETARIO: todos los campos incluyendo diagnóstico y evaluaciones
- TESORERO: columnas reducidas (sin datos clínicos sensibles)

Estado Actual del Proyecto

Módulo	Estado
auth	✅ Implementado y testeado
users	✅ Implementado y testeado
geo	✅ Implementado y testeado
ataxia-types	✅ Implementado y testeado
miembros	✅ Implementado y testeado
diagnostico-clinico	✅ Implementado y testeado

Módulo	Estado
evaluacion-funcional	✓ Implementado (append-only) y testeado
stats	✓ Implementado (solo lectura, sin entidad propia) y testeado
audit	✓ Implementado (append-only) y testeado
exports	✓ Implementado (CSV/XLSX/PDF) y testeado
Swagger	✓ Configurado en todos los módulos implementados
Docker Compose	✓ PostgreSQL 16 local (puerto 5434)
Seguridad	✓ helmet + throttler configurados

Próximos Pasos

1. Implementar módulo `cuotas` (TarifaAnual + Cuota, diseño documentado) y además todos los módulos que sean parte de finanzas queda para otras etapas de este proyecto por decisión de la directiva de la asociación.
2. Migraciones TypeORM para paso a producción
3. Tests e2e para módulos de diagnóstico, evaluación y stats

AtaxChile Backend

Repositorio: `backend/`

Documentación técnica: `docs/api-design.md`

Guía para agentes IA: `AGENTS.md`

Swagger UI: `http://localhost:5000/api/docs`

Agrupación de Pacientes con Ataxia – Chile